

From Authenticated Encryption to Hash: A Comprehensive Design Space Exploration of the NIST Standard Ascon Family

Muhammad Sohaib Munir
Tallinn University of Technology
Tallinn, Estonia
muhmun@taltech.ee

Levent Aksoy
Tallinn University of Technology
Tallinn, Estonia
levent.aksoy@taltech.ee

Tommaso Dordoni
ETH Zurich
Zurich, Switzerland
tdordoni@student.ethz.ch

Sedat Akleylek
University of Tartu
Tartu, Estonia
sedat.akleylek@ut.ee

Abstract

Ascon is the National Institute of Standards and Technology (NIST) standard for lightweight cryptography, providing Authenticated Encryption with Associated Data (AEAD), hash, eXtendable-Output Function (XOF), and Customizable XOF (CXOF). Prior studies mainly target AEAD implementations of earlier Ascon versions with a little focus on hash and unified designs, and generally explore the area and latency tradeoff by varying permutation rounds per one clock cycle. In this work, we present a comprehensive design space exploration of the complete Ascon family of the NIST standard for the first time through two design architectures: (i) the conventional round-based variants that perform 1, 2, and 4 rounds per one clock cycle, i.e., *v1r1c*, *v2r1c*, and *v4r1c*, respectively, and (ii) the operation-based variants that execute one 64-bit and 32-bit operation per one clock cycle using a cycle-optimized schedule that completes one permutation round with a total of 28 and 56 cycles, i.e., *v1op1c_64b_28* and *v1op1c_32b_56*, respectively. Our design space exploration includes the Ascon-AEAD128 encryption, decryption, unified encryption/decryption, unified Ascon-Hash256, Ascon-XOF128, Ascon-CXOF128, and a complete Ascon unified family. Experimental results show that the round-based variants achieve low latency and energy consumption, while the operation-based variants lead to designs with low area. Our implementations also have smaller hardware complexity than the state-of-the-art designs. For example, on the Ascon-AEAD128 designs performing encryption, our round-based *v1r1c* variant achieves an area reduction of 39.99% when compared to a round-based state-of-the-art design, while our *v1op1c_64b_28* variant reduces latency and energy consumption by 48.11% and 32.29%, respectively when compared to the state-of-the-art design using one 64-bit operation per one clock cycle, completing one permutation round in 59 clock cycles.

CCS Concepts

• **Hardware** → **Very large scale integration design**; • **Security and privacy** → **Cryptography**; *Security in hardware*.



This work is licensed under a Creative Commons Attribution 4.0 International License.
GLSVLSI '26, Canandaigua, NY, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2431-2/26/06
<https://doi.org/10.1145/3787109.3815232>

Keywords

Lightweight cryptography, authenticated encryption, hash function, NIST standard Ascon family, design space exploration.

ACM Reference Format:

Muhammad Sohaib Munir, Tommaso Dordoni, Levent Aksoy, and Sedat Akleylek. 2026. From Authenticated Encryption to Hash: A Comprehensive Design Space Exploration of the NIST Standard Ascon Family. In *Great Lakes Symposium on VLSI 2026 (GLSVLSI '26)*, June 22–24, 2026, Canandaigua, NY, USA. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3787109.3815232>

1 Introduction

In today's Internet of Things (IoT) world, connected edge devices are widely deployed across smart homes and buildings, industrial automation, healthcare, and transportation. As these devices often operate on sensitive data under constrained area and energy resources, they require strong security with a small overhead. Consequently, lightweight cryptography has gained prominence with block ciphers, such as SIMON [1] and PRESENT [2], demonstrating hardware-efficient designs. Recently, the National Institute of Standards and Technology (NIST) selected the Ascon family as the lightweight cryptography standard, covering Authenticated Encryption with Associated Data (AEAD), hash, eXtendable-Output Function (XOF), and Customizable XOF (CXOF), i.e., Ascon-AEAD128, Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128, respectively [19]. Earlier versions of Ascon were submitted to the Competition for Authenticated Encryption: Security, Applicability, and Robustness (CAESAR), including Ascon v1 [3], Ascon v1.1 [4], and Ascon v1.2 [5]. The CAESAR authenticated encryption candidates were Ascon-128 and Ascon-128a. The subsequent NIST submission [6] expands the family to include the AEAD variants Ascon-128, Ascon-128a, and Ascon-80pq, as well as hash functions, namely Ascon-Hash and Ascon-Hasha, and the XOFs, i.e., Ascon-XOF and Ascon-XOFA.

Previous studies, such as [5, 10, 13, 15], have examined the design space of previous versions of the Ascon family, but to the best of our knowledge, such an analysis has not yet been performed for the most recent NIST-standardized version. Most of these studies [5, 13, 15] concentrated on architectural approaches that adjust the number of permutation rounds executed per one clock cycle. The design architectures that offer a low-area design are very limited and use a large number of clock cycles [10]. Additionally, the

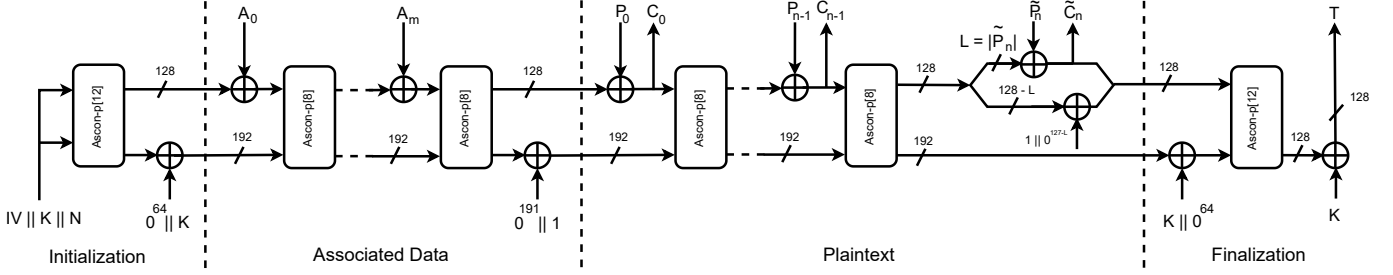


Figure 1: Encryption in Ascon-AEAD128.

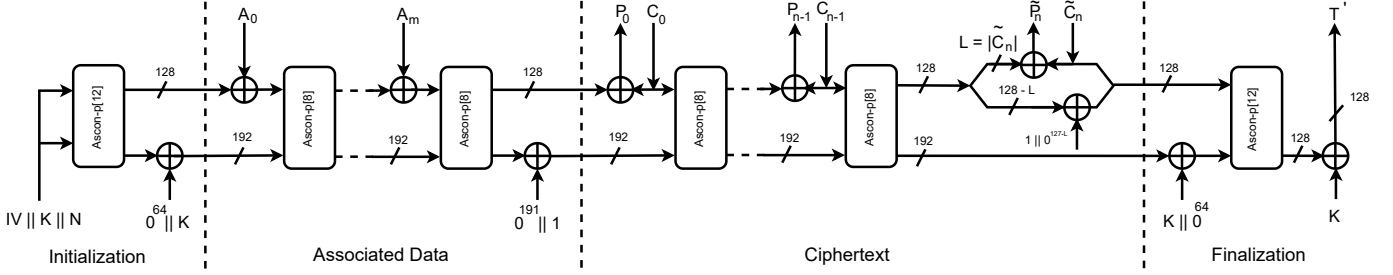


Figure 2: Decryption in Ascon-AEAD128.

Table 1: Lookup table of the substitution layer.

X	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
Y	04	0b	1f	14	1a	15	09	02	1b	05	08	12	1d	03	06	1c
X	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e	1f
Y	1e	13	07	0e	00	0d	11	18	10	0c	01	19	16	0a	0f	17

majority of existing implementations focused on authenticated encryption [5, 10, 15, 17], while comparatively fewer works investigated hash function implementations [7, 13] or unified designs capable of supporting multiple Ascon functionalities [8, 18, 20]. The Ascon unified family designs neither include XOFs nor provide a design space exploration. Hence, we provide a broader exploration of the NIST standard Ascon family with the following contributions:

- Design architectures including the round-based variants that perform 1, 2, and 4 rounds per one clock cycle in the permutation block, i.e., *v1r1c*, *v2r1c*, and *v4r1c*, respectively, and the operation-based variants that execute one 64-bit and 32-bit operation per one clock cycle completing one permutation round in a minimum number of 28 and 56 clock cycles, i.e., *v1op1c_64b_28* and *v1op1c_32b_56*, respectively;
- Design space exploration of the AEAD design including encryption, decryption, and unified encryption/decryption;
- Design space exploration of a unified hash design including Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128;
- Design space exploration of the Ascon unified family design supporting Ascon-AEAD128 encryption/decryption, Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128.

Experimental results show that the proposed design architectures lead to alternative designs, exploring the tradeoff between area, latency, and energy consumption. While the round-based variants have small latency and energy consumption, the operation-based variants occupy small area. For example, in the Ascon-AEAD128 encryption design, the operation-based variant *v1op1c_64b_28* reduces area by 32.82% when compared to the round-based variant *v1r1c*, but increases the latency and energy consumption by 63.97×

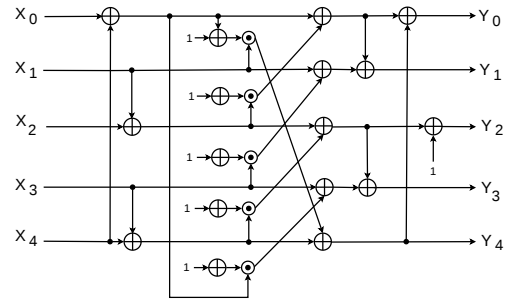


Figure 3: Circuit design of the lookup table.

and 58.75×, respectively. The Ascon unified family design implementing five functions incurs only a slight overhead ranging from 1.76% to 5.2% when compared to the unified AEAD design.

The rest of this paper is organized as follows: Section 2 presents the NIST standard Ascon family and related work. The proposed design architectures are described in Section 3. Experimental results are presented in Section 4. Finally, Section 5 concludes the paper.

2 Background

2.1 Ascon AEAD Function

Figure 1 illustrates the encryption process in Ascon-AEAD128 [19], which has four main stages: *Initialization*, *Associated Data*, *Plaintext*, and *Finalization*. In the *Initialization* stage, the 320-bit state is generated by concatenating a 64-bit initialization vector *IV*, a 128-bit key *K*, and a 128-bit nonce *N*. This 320-bit state enters into the permutation block, which runs 12 rounds. The output after the *Initialization* stage is passed to the *Associated Data* stage, where a variable-length associated data ($A_0, \dots, A_m$) is processed in 8 permutation rounds. In the third stage *Plaintext*, 8 permutation rounds are applied to the variable-length plaintext ($P_0, \dots, P_n$) and the ciphertext ($C_0, \dots, C_n$) is produced. The last stage is *Finalization*,

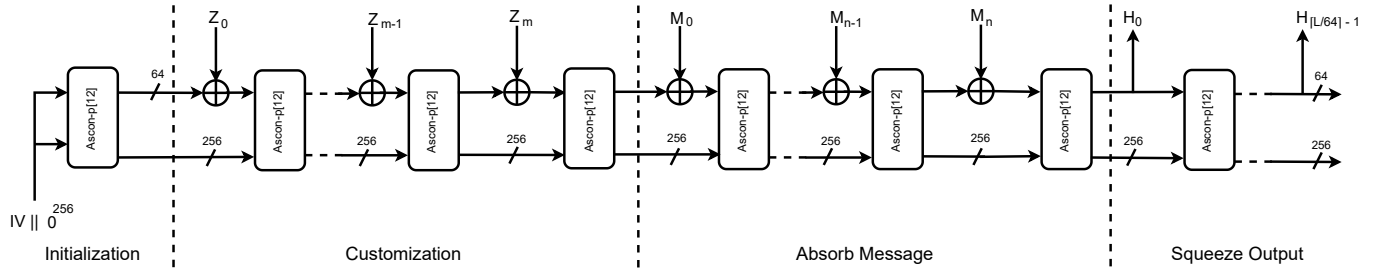


Figure 4: Structure of Ascon-CXOF128.

where 12 permutation rounds are applied, and the 128-bit authentication tag T is generated. Its decryption process is given in Figure 2. The decryption process follows the same stages as encryption. The main differences appear in the *Ciphertext* stage, which consists of regular and last block processing. In the regular block processing, each plaintext is XORed with the current state to produce the updated state during encryption, whereas the state is updated with the ciphertext without performing the XOR operation during decryption. In the last block processing, the final plaintext, including the padded bits, is XORed with the state during encryption, whereas the ciphertext is directly inserted into the state and the padded bits are XORed with the remaining state bits during decryption.

As evident from Figures 1–2, the core component is the permutation block. It comprises three layers: *constant-addition*, *substitution*, and *linear diffusion*. In this block, the 320-bit state is considered as five 64-bit state words (S_0, S_1, S_2, S_3, S_4). In the *constant-addition* layer, a 64-bit round constant and S_2 are added using XOR gates. The *substitution* layer updates all five 64-bit state words using the lookup table given in Table 1. Figure 3 presents the design of this lookup table using XOR and AND gates. The *linear diffusion* layer applies circular right rotations and XOR operations as given in Eq. (1) to disperse information across the states.

$$\begin{aligned}
 S_0 &\leftarrow S_0 \oplus (S_0 \ggg 19) \oplus (S_0 \ggg 28), \\
 S_1 &\leftarrow S_1 \oplus (S_1 \ggg 61) \oplus (S_1 \ggg 39), \\
 S_2 &\leftarrow S_2 \oplus (S_2 \ggg 1) \oplus (S_2 \ggg 6), \\
 S_3 &\leftarrow S_3 \oplus (S_3 \ggg 10) \oplus (S_3 \ggg 17), \\
 S_4 &\leftarrow S_4 \oplus (S_4 \ggg 7) \oplus (S_4 \ggg 41).
 \end{aligned} \tag{1}$$

2.2 Ascon Hash Function

Ascon-CXOF128 follows a sponge-based structure with four stages, *Initialization*, *Customization*, *Absorb Message*, and *Squeeze Output*, as illustrated in Figure 4. First, the 320-bit internal state is initialized by concatenating the 64-bit IV with 256 zero bits. In the *Customization* stage, the customization string ($Z_0, \dots, Z_m$) is incorporated into the state prior to absorbing the message. During the message absorption, the variable-length message ($M_0, \dots, M_n$) is processed as 64-bit blocks. Finally, the output is generated by squeezing as many 64-bit blocks as required to produce L bits, which corresponds to $\lceil L/64 \rceil$ output blocks ($H_0, \dots, H_h$), where $h = \lceil L/64 \rceil - 1$.

Ascon-XOF128 follows the same structure as Ascon-CXOF128, but omits the *Customization* stage. In this case, after *Initialization*, the message is directly absorbed into the state.

Ascon-Hash256 follows the same sponge-based structure as Ascon-XOF128, except the digest is generated by squeezing four

64-bit output blocks from the 320-bit state, where h is 3. These four 64-bit blocks are then concatenated to form the final 256-bit digest.

Note that the IV is different for Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128, reflecting the distinct hash function.

2.3 Related Works

The existing hardware implementations and design space exploration studies of Ascon focus on its earlier versions, and to the best of our knowledge, there exists no design space exploration of the NIST standard Ascon family.

The application of Ascon-128 and Ascon-128a for AI-enabled IoT devices is explored in [15]. For each variant, two different architectures are proposed: (i) a recursive version and (ii) an unrolled version. The recursive design executes two permutation rounds per one clock cycle, while the unrolled design completes all of the rounds in a single clock cycle. Similarly, a broader design space exploration of Ascon-128 and Ascon-128a is presented in [14]. This study considers loop-folded architectures executing one permutation round per one clock cycle, loop-unrolled architectures executing multiple rounds per one clock cycle (two, three, or six rounds for Ascon-128 and two or four rounds for Ascon-128a), as well as fully unrolled architectures, in which all permutation rounds are completed within a single clock cycle. FPGA implementations of Ascon-128 and Ascon-128a aimed at improving the throughput-to-area ratio are presented in [17]. To achieve this goal, designs with six permutation rounds per one clock cycle for Ascon-128 and four rounds per one clock cycle for Ascon-128a are implemented. Notably, most designs in the literature focus on the number of the permutation rounds per one clock cycle, with the exception of [10], where apart from implementing the Ascon-fast designs, which can execute the permutation block in one, two, three, and six rounds per one clock cycle, the Ascon-64-bit and Ascon-x-low-area designs are implemented. While the former uses 64-bit operations, the latter uses 1-bit operations. These designs are based on Ascon v1.

The design space exploration of the Ascon-Hash function is conducted by performing one, two, four, and six permutation rounds per one clock cycle in [13]. In contrast, RECO-ASCON [7] presents a reconfigurable Ascon hashing processor that supports both Ascon-Hash and Ascon-Hasha in a single compact design, implemented in Chisel for IoT applications.

The work of [9] proposes a programmable Ascon AEAD hardware core that supports all three AEAD variants, namely Ascon-128, Ascon-128a, and Ascon-80pq, within a single unified design, avoiding the need for separate standalone implementations. Extending this direction beyond AEAD-only support, the work of [8] presents

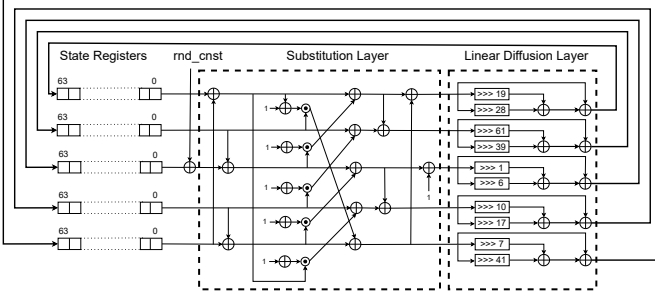


Figure 5: Permutation block of the *v1r1c* variant.

an optimized programmable Ascon core that integrates both AEAD (Ascon-128/128a/80pq) and hash (Ascon-Hash and Ascon-Hasha) within the same architecture. Similarly, the work of [20] introduces RECO-HCON, a high-throughput, reconfigurable, and compact Ascon processor with six operating modes, combining AEAD (Ascon-128/128a) and hash (Ascon-Hash/Hasha) in a single design. The design and integration of Ascon-128 and Ascon-Hash functions in an open-source RISC-V based processor are given in [18]. A single architecture is used for both designs, and switching between the AEAD and hash functions is performed with control signals. The RISC-V core used for integration is SERV-core [16], and Ascon is integrated with the Wishbone Interface.

3 Design of the NIST Standard Ascon Family

This section describes the design of the NIST standard Ascon family. We first present the design architectures proposed for the permutation block. Then, we describe the unified designs that support AEAD encryption/decryption, Hash256/XOF128/CXOF128, and the complete Ascon family, reusing a single permutation block.

3.1 Design Architectures

This section presents the proposed design architectures for the fundamental block of the Ascon family, the permutation block.

3.1.1 Round-Based Variants. We present three round-based variants that execute: (i) one permutation round per one clock cycle *v1r1c*, (ii) two permutation rounds per one clock cycle *v2r1c*, and (iii) four permutation rounds per one clock cycle *v4r1c*. Figure 5 shows the permutation block of the *v1r1c* design, in which exactly one round is executed per one clock cycle. In each round, five 64-bit state words are read from the state registers, the *constant-addition*, *substitution*, and *linear diffusion* layers are computed, and the updated state words are stored in state registers for the next round. The *v2r1c* and *v4r1c* design architectures replicate the *constant-addition*, *substitution*, and *linear diffusion* layers twice and four times, respectively, when compared to the *v1r1c* design architecture. The permutation rounds per one clock cycle are chosen as 1, 2, and 4 in the AEAD design because the *Initialization* and *Finalization* stages require 12 rounds, while the *Associated Data* and *Plaintext/Ciphertext* stages require 8 rounds, and 1, 2, and 4 are the common divisors of 12 and 8. Additionally, for the unified hash designs, we introduce the *v6r1c* variant, which can execute six permutation rounds per one clock cycle, since all stages use 12 permutation rounds. In this way, the same permutation block can be reused across all stages, reducing the total area.

The number of clock cycles required to complete the encryption/decryption process in Ascon-AEAD128 is given as follows:

$$\begin{aligned} C_{\text{AEAD}} &= \frac{12}{r} + \frac{8M}{r} + \frac{8(N-1)}{r} + \frac{12}{r} + C_{\text{ovh}} \\ &= \frac{16 + 8(M+N)}{r} + C_{\text{ovh}} \\ C_{\text{ovh}} &= 2 + 2M + 2N + 1 = 3 + 2(M+N) \end{aligned} \quad (2)$$

where $r \in \{1, 2, 4\}$ denotes the number of permutation rounds completed in one clock cycle, M and N are the number of 128-bit processed associated-data and plaintext/ciphertext blocks, respectively, and C_{ovh} captures the implementation-dependent overhead due to the initial state construction, the transitions between stages, new input-block insertion, and output handling.

Similarly, the number of clock cycles required to complete the hash process in the Ascon family is given as follows:

$$\begin{aligned} C_{\text{Hash}} &= \frac{12}{r} + \frac{12C}{r} + \frac{12K}{r} + \frac{12(D-1)}{r} + C_{\text{ovh}} \\ &= \frac{12(C+K+D)}{r} + C_{\text{ovh}} \\ C_{\text{ovh}} &= 2 + 2C + 2K + \max(1, 3D-3) \\ &= 2(1+C+K) + \max(1, 3D-3) \end{aligned} \quad (3)$$

where $r \in \{1, 2, 4, 6\}$, and C , K , and D denote the number of 64-bit customization string, message, and digest blocks, respectively. Note that the term $\max(1, 3D-3)$ in C_{ovh} denotes the output handling, which requires 2, 3, and 1 clock cycles for the first, intermediate, and last digest block, respectively. Hence, if D is 1, this overhead is 1, otherwise it is $3D-3$.

Note that the first four addends in the unsimplified forms of Eqs. (2) and (3) correspond to the number of clock cycles in the four stages of AEAD (Figures 1 and 2) and hash (Figure 4) functions, respectively.

3.1.2 Operation-Based Variants. In order to reduce the hardware area with respect to the round-based designs, registers and operations can be shared, taking into account an increase in latency and energy consumption. Hence, we present two operation-based variants that execute: (i) one 64-bit operation per one clock cycle, completing one permutation round in 28 cycles *v1op1c_64b_28* and (ii) one 32-bit operation per one clock cycle, completing one permutation round in 56 cycles *v1op1c_32b_56*. Both design architectures use an Arithmetic Logic Unit (ALU) that supports XOR and NOT-AND¹ operations, and performs one operation per one clock cycle. The design uses only seven registers: five to store five 64-bit state words (S_0, S_1, S_2, S_3, S_4) and two to store temporary computations (T_0, T_1)².

To implement one operation per one clock cycle using the minimum number of clock cycles, the computations in the *substitution* and *linear diffusion* layers are modified for the *v1op1c_64b_28* design architecture as given in the following.

Substitution Layer: We introduce a set of operations that requires seven registers, five permanent and two temporary, and

¹This two-input operation $y = \text{NOT-AND}(a, b)$ occurs in the circuit design of the look-up table given in Figure 3 as $y = (1 \oplus a) \odot b$.

²The number of temporary registers is minimum since the maximum fanout of operations given in Figure 3 is 3.

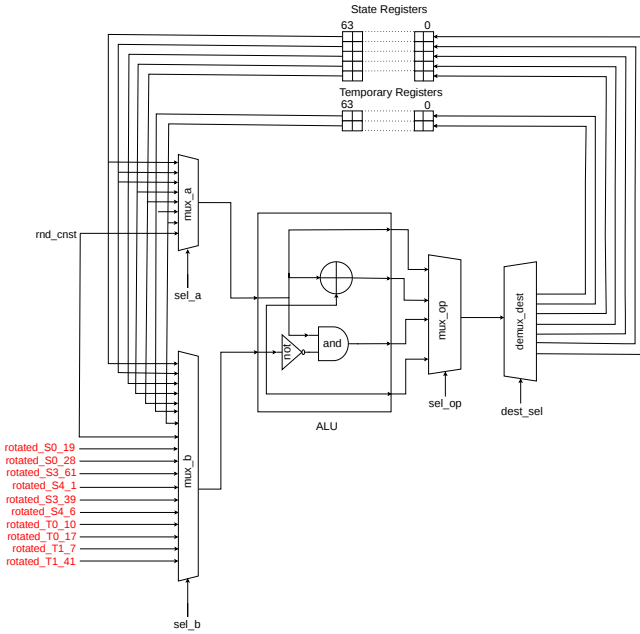


Figure 6: Permutation block of the *v1op1c_64b_28* variant.

implements the computations of the *substitution* layer given in Figure 3 as follows:

$$\begin{aligned}
 t0 : S_0 &= S_0 \wedge S_4 & t9 : T_0 &= S_4 \wedge T_0 \\
 t1 : S_2 &= S_1 \wedge S_2 & t10 : S_3 &= \sim S_2 \& S_3 \\
 t2 : S_4 &= S_3 \wedge S_4 & t11 : S_3 &= S_1 \wedge S_3 \\
 t3 : T_0 &= \sim S_4 \& S_0 & t12 : S_2 &= \sim S_1 \& S_2 \\
 t4 : T_1 &= \sim S_0 \& S_1 & t13 : S_2 &= S_0 \wedge S_2 \\
 t5 : T_1 &= S_4 \wedge T_1 & t14 : S_0 &= S_2 \wedge T_1 \\
 t6 : S_4 &= \sim S_3 \& S_4 & t15 : S_2 &= S_2 \wedge S_3 \\
 t7 : S_4 &= S_2 \wedge S_4 & t16 : S_4 &= S_4 \wedge \{64\{1'b1\}\} \\
 t8 : T_0 &= S_3 \wedge T_0 & &
 \end{aligned} \tag{4}$$

Note that the substitution layer is completed in 17 cycles, which is the minimum achievable cycle count since this layer includes 17 XOR and NOT-AND operations as given in Figure 3, thereby reducing the *substitution* layer latency by 8 cycles when compared to the state-of-the-art design [10].

Linear Diffusion Layer: Since all shift amounts in the *linear diffusion* layer are fixed constants as shown in Eq. (1), we precompute the required rotated versions of each 64-bit state word. These precomputed rotations are then selected through the multiplexor and connected to the XOR operation. Thus, the computations for each 64-bit state word can be completed in 2 clock cycles, a total of 10 clock cycles for all computations in the *diffusion* layer, thereby reducing the *diffusion* layer latency by 23 clock cycles when compared to the state-of-the-art design [10].

Figure 6 illustrates the permutation block of the *v1op1c_64b_28* design architecture. Combined with the optimizations in the number of clock cycles in the *substitution* and *diffusion* layers, the proposed architecture realizes the permutation block in 28 clock cycles when compared to 59 in the state-of-the-art design [10]. The *v1op1c_32b_56* variant reuses the same architecture as the *v1op1c_64b_28* variant, but all 64-bit state words and temporary

registers are processed in two 32-bit slices, and 32-bit XOR and NOT-AND operations are used. Therefore, the *substitution* and *diffusion* layers remain unchanged. However, each 64-bit operation requires two cycles, increasing the number of clock cycles required to realize the computations in the permutation block to 56.

The number of clock cycles required to complete the encryption/decryption process in Ascon-AEAD128 is given as follows:

$$\begin{aligned}
 C_{\text{AEAD}} &= (28w \times 12) + (28w \times 8)M \\
 &\quad + (28w \times 8)(N - 1) + (28w \times 12) + C_{\text{ovh}} \\
 &= w(224M + 224N + 448) + C_{\text{ovh}} \\
 C_{\text{ovh}} &= (1 + 7w) + (M(2w + 1) + 1) + N(4w + 1) + 4w \\
 &= w(2M + 4N + 11) + (M + N + 2)
 \end{aligned} \tag{5}$$

where $w \in \{1, 2\}$ denotes the *v1op1c_64b_28* and *v1op1c_32b_56* variants when w is set to 1 and 2, respectively. Similarly, the number of clock cycles required to complete the hash process in the Ascon family is given as follows:

$$\begin{aligned}
 C_{\text{Hash}} &= (28w \times 12) + (28w \times 12)C \\
 &\quad + (28w \times 12)K + (28w \times 12)(D - 1) + C_{\text{ovh}} \\
 &= 336w(C + K + D) + C_{\text{ovh}} \\
 C_{\text{ovh}} &= (1 + 5w) + C(w + 1) + K(w + 1) + (1 + wD) \\
 &= w(5 + C + K + D) + (2 + C + K).
 \end{aligned} \tag{6}$$

3.2 Unified Designs

The Ascon-AEAD128 encryption and decryption processes discussed in Section 2.1 share the same stages and differ only in the *Plaintext/Ciphertext* stage, which enables a unified design. The proposed design uses a single permutation block and a main control logic for both modes, while restricting mode-dependent behavior to the *Plaintext/Ciphertext* stage in the unified design. In particular, the control logic determines how the incoming text block is combined with the 320-bit state in the *Plaintext/Ciphertext* stage. This design choice supports both encryption and decryption processes with a small additional logic while maximizing hardware reuse.

A similar resource-sharing approach is used for the hash functions. As described in Section 2.2, Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128 have a similar sponge-based structure, which enables a unified hardware architecture to support all three functions within a single design. The unified design reuses one permutation block and the same datapath across three modes, selecting the appropriate IV and adding the *Customization* stage before the *Absorb Message* stage for CXOF. To keep the output interface compact, a single 64-bit output register is used to store the digest block sequentially during the *Squeeze Output* stage, allowing fixed-length Hash256 (four 64-bit blocks) and variable-length XOF/CXOF outputs to be produced using the same output mechanism.

Building on the unified encryption/decryption and hash designs, a single Ascon unified family design is constructed to support all five functions of the Ascon family. The design reuses the same permutation block and major datapath resources across the supported modes, while additional multiplexing is introduced for IV selection, input insertion, and output handling.

Table 2: Synthesis results of the Ascon-AEAD128 design.

Function	Type	Variant	Area (μm^2)	Delay (ps)	Cycles	Latency (ns)	Power (μW)	Energy (pJ)
Encryption	Round-based	<i>v1r1c</i>	17343	1112	59	66	10842	711
		<i>v2r1c</i>	22330	1702	35	60	21140	1259
		<i>v4r1c</i>	31859	3085	23	71	57788	4100
	Operation-based	<i>v1op1c_64b_28</i>	11651	3075	1373	4222	9894	41772
		<i>v1op1c_32b_56</i>	11721	2765	2740	7576	6187	46873
Decryption	Round-based	<i>v1r1c</i>	18427	1298	59	77	10902	835
		<i>v2r1c</i>	23538	1702	35	60	21307	1269
		<i>v4r1c</i>	33066	3085	23	71	57964	4094
	Operation-based	<i>v1op1c_64b_28</i>	11968	2865	1373	3934	9187	36138
		<i>v1op1c_32b_56</i>	12099	2967	2740	8130	6289	51127
Unified Encryption/Decryption	Round-based	<i>v1r1c</i>	17902	1138	59	67	10972	737
		<i>v2r1c</i>	23053	1702	35	60	21276	1267
		<i>v4r1c</i>	32581	3085	23	71	57914	4109
	Operation-based	<i>v1op1c_64b_28</i>	11977	2795	1373	3838	9663	37082
		<i>v1op1c_32b_56</i>	12098	3037	2740	8321	6308	52491

Table 3: Synthesis results of unified Ascon-AEAD128 encryption/decryption design with minimum critical-path delay.

Variant	Area (μm^2)	Delay (ps)	Frequency (MHz)	Cycles	Latency (ns)	Power (μW)	Energy (pJ)
<i>v1r1c</i>	22254	503	1988	59	30	13464	400
<i>v2r1c</i>	33606	809	1236	35	28	37965	1075
<i>v4r1c</i>	60715	1413	708	23	32	168335	5470
<i>v1op1c_64b_28</i>	16376	874	1144	1373	1200	9475	11370
<i>v1op1c_32b_56</i>	17307	928	1078	2740	2543	8183	20807

4 Experimental Results

In this section, we present the gate-level synthesis results of the NIST standard Ascon family, including encryption, decryption, unified encryption/decryption processes in Ascon AEAD128, unified hash functions Ascon-Hash256, Ascon-XOF128, and Ascon-CXOF128, and the complete Ascon unified family.

All the design architectures are described at the register-transfer level (RTL) in Verilog. The gate-level synthesis results are obtained with Cadence Genus after logic synthesis, targeting area optimization, using a commercial 65 nm standard-cell library. All variants are verified through simulation using randomly generated 15000 test vectors, where the obtained switching activity is used to compute the power dissipation.

4.1 Ascon AEAD Function

Table 2 reports the gate-level synthesis results of designs implementing the Ascon-AEAD128 encryption, decryption, and unified encryption/decryption functions. In this table, *Area* indicates the total area. *Delay* is the critical path delay. *Cycles* is the number of clock cycles required to produce the authentication tag, computed for 128-bit associated data and plaintext by setting *M* and *N* to 2 in Eq. (2) and Eq. (5) for the round-based and operation-based design architectures, respectively. *Latency* is computed as the multiplication of the number of clock cycles by delay. *Power* denotes the total power dissipation. *Energy* is the energy consumption computed as the multiplication of the latency by the power dissipation.

Observe from Table 2 that increasing the permutation rounds per one clock cycle from 1 to 2 and 4 in the round-based variants increases area by 1.29 $\times$ and 1.84 $\times$, respectively for the encryption process. In return, both round-based variants complete in fewer cycles than the *v1r1c* variant. While the *v2r1c* variant reduces the latency by 9.09%, the *v4r1c* variant increases the latency by 7.58% due to its larger delay when compared to the *v1r1c* variant. Compared to the *v1r1c* variant, the energy consumption increases by 1.77 $\times$ and 5.77 $\times$ in the *v2r1c* and *v4r1c* variants, respectively, primarily due to

their higher power dissipation. When the *v1op1c_64b_28* variant is compared with the *v1r1c* variant for the encryption process, the area decreases by 32.82%, while the latency and energy consumption increase by 63.97 $\times$ and 58.75 $\times$, respectively. The *v1op1c_32b_56* variant similarly reduces area by 32.42% when compared to the *v1r1c* variant, increasing the latency and energy consumption by 114.79 $\times$ and 65.93 $\times$, respectively. It can be observed from Table 2 that the decryption process incurs only a slight area overhead when compared to the encryption process in all variants as mentioned in Section 2.1. The *v1r1c* variant in the decryption process exhibits a 6.25% increase in area relative to the *v1r1c* variant in the encryption process. Compared to implementing encryption and decryption in separate designs, the unified design achieves dual functionality with a little overhead in area when compared to the design architectures in the encryption process. It is interesting to note that the delay of the *v2r1c* and *v4r1c* variants remain the same for all functions because the critical path of these designs is determined as the one in their permutation block. Similarly, the round-based variants for the unified encryption/decryption design achieves slightly lower area than the corresponding variants in the decryption design. This can be attributed to effective logic sharing and optimization of the common datapath and control logic in the unified encryption/decryption design.

Table 3 reports the results of the unified encryption/decryption designs at their minimum (maximum) critical-path delay (frequency). Each variant is resynthesized iteratively with a delay constraint/determined in a binary search manner until the smallest achievable delay is reached.

Observe from Table 3 that the round-based variant *v1r1c* incurs an area overhead of 24.31% when compared to the *v1r1c* variant for the unified encryption/decryption process given in Table 2. In return, it achieves substantially low latency and energy consumption: latency decreases by 55.22% while energy consumption decreases by 45.73%. Note that the maximum frequency in this design is increased 2.26 $\times$, reaching to 1.99 GHz. These improvements stem primarily from the minimum critical-path delay. Similar trends are observed for the other round-based variants, except the *v4r1c* variant, where the energy consumption increases due to the increase in power dissipation, and the operation-based variants. The *v1r1c* design has the shortest critical-path because, as shown in Figure 5, it performs one permutation round in one clock cycle.

Table 4: Synthesis results of the unified Ascon Hash design (Ascon-Hash256 / Ascon-XOF128 / Ascon-CXOF128).

Type	Variant	Area (μm^2)	Delay (ps)	Cycles		Latency (ns)		Power (μW)	Energy (pJ)	
				HASH256 XOF128	CXOF128	HASH256 XOF128	CXOF128		HASH256 XOF128	CXOF128
Round-based	<i>v1r1c</i>	12694	1713	87	129	149	221	8996	1341	1988
	<i>v2r1c</i>	17580	1709	51	75	87	128	17961	1565	2302
	<i>v4r1c</i>	27234	2721	33	48	90	131	49966	4487	6526
	<i>v6r1c</i>	36493	4129	27	39	111	161	109799	12188	17678
Operation-based	<i>v1op1c_64b_28</i>	8871	2364	2031	3045	4801	7198	5908	28366	42528
	<i>v1op1c_32b_56</i>	9004	3246	4058	6083	13172	19745	5266	69365	103979

Table 5: Synthesis results of the Ascon unified family design.

Type	Variant	Area (μm^2)	Delay (ps)	Cycles			Latency (ns)			Power (μW)	Energy (pJ)		
				AEAD	HASH256 XOF128	CXOF128	AEAD	HASH256 XOF128	CXOF128		AEAD	HASH256 XOF128	CXOF128
Round-based	<i>v1r1c</i>	18835	1697	59	87	129	100	148	219	11068	1108	1634	2423
	<i>v2r1c</i>	23517	1770	35	51	75	62	90	133	20671	1281	1866	2744
	<i>v4r1c</i>	33156	3068	23	33	48	71	101	147	54973	3879	5566	8096
	<i>v1op1c_64b_28</i>	12340	2907	1373	2031	3045	3991	5904	8852	9123	36413	53863	80755
Operation-based	<i>v1op1c_32b_56</i>	12464	2996	2740	4058	6083	8209	12158	18225	6386	52423	77640	116383

Table 6: Round-based state-of-the-art design comparison.

Variant	Design	Area (μm^2)	Delay (ps)	Cycles	Latency (ns)	Power (μW)	Energy (pJ)
<i>v1r1c</i>	[14]	28902	1795	47	84	20308	1713
	Ours	17343	1112	59	66	10842	711
<i>v2r1c</i>	[14]	33519	2108	27	57	36353	2069
	Ours	22330	1702	35	60	21140	1259
<i>v4r1c</i>	[14]	42265	3130	17	53	68843	3663
	Ours	31859	3085	23	71	57788	4100

4.2 Ascon Hash Function

Table 4 presents the results of the unified Ascon hash design under the round-based and operation-based design architectures. The number of cycles is computed when 64-bit message and 256-bit digest are used for Ascon-Hash256 and Ascon-XOF128, setting the values of C , K , and D to 0, 2, and 4, respectively, in Eqs. (3) and (6). It is computed when 64-bit customization data, 64-bit message, and 256-bit digest are used for Ascon-CXOF128, setting the values of C , K , and D to 3, 2, and 4, respectively, in Eqs. (3) and (6) for the round-based and operation-based design architectures, respectively.

As also observed for the Ascon-AEAD128 designs in Table 2, it can be seen from Table 4 that the operation-based variants offer smaller area than the round-based variants. For instance, in the Ascon-Hash256 and Ascon-XOF128 designs, the *v1op1c_64b_28* variant reduces the area by 30.12%, when compared to the *v1r1c* variant. In contrast, the round-based design architectures achieve lower latency and energy consumption, offering 55.18× lower latency in the *v2r1c* variant and 21.15× lower energy consumption in the *v1r1c* variant when compared to the *v1op1c_64b_28* variant.

4.3 Ascon Unified Family

Table 5 presents the gate-level synthesis results of the Ascon unified family design. Note that the number of cycles is computed when the same size of data is used in the unified encryption/decryption and unified hash designs as given in Section 4.1 and 4.2, respectively.

Observe from Table 5 that when compared to the round-based implementations of the unified encryption/decryption using the *v1r1c*, *v2r1c*, and *v4r1c* variants given in Table 2, the area increases by only 5.2%, 2.0%, and 1.76%, respectively. Similarly, the operation-based variants of the Ascon unified family incur area overheads of 3.0% for the *v1op1c_64b_28* and *v1op1c_32b_56* variants when compared to the corresponding implementations in Table 2. In terms of latency, the round-based variant *v1r1c* in the Ascon unified family

incurs a 49.25% overhead compared to the corresponding unified encryption/decryption *v1r1c* implementation in Table 2. This increase is due to the critical path delay. A latency overhead of 3.3% is observed for the *v2r1c* variant, while the *v4r1c* variant incurs no latency overhead due to a slight increase in the delay. Similarly, the operation-based variant *v1op1c_64b_28* in the Ascon unified family introduces a latency overhead of 4.0% when compared to that in the unified encryption/decryption given in Table 2. In contrast, the *v1op1c_32b_56* variant exhibits lower latency than its unified encryption/decryption counterpart in Table 2, owing to a reduced critical-path delay. Regarding energy consumption, the round-based variant *v1r1c* in the Ascon unified family exhibits a 50.3% increase compared to the corresponding unified encryption/decryption implementation in Table 2. The *v2r1c* variant increases energy consumption by 1.1%. In contrast, the *v4r1c* variant demonstrates lower energy consumption due to reduced power dissipation. Similarly, the operation-based variants exhibit reduced energy consumption relative to the corresponding unified encryption/decryption design in Table 2 due to lower power dissipation.

When compared with the Ascon unified hash design in Table 4, it can be observed that both round-based and operation-based variants incur higher area and energy consumption. This is simply because of the additional AEAD-specific control and datapath logic, as well as the need for two 128-bit registers in the Ascon unified family (one for text and one for the authentication tag), whereas the Ascon unified hash design uses only a single 64-bit output register.

In summary, the Ascon unified design implements 5 functions, including encryption, decryption, Hash256, XOF128, and CXOF128, with a little area overhead with respect to the unified encryption/decryption design.

4.4 Comparison with the State-of-the-Art

For the state-of-the-art design comparison, we used the publicly available RTL implementations in [11] and [12], corresponding to the hardware implementations in [14] and [10], respectively and synthesized them using the same logic synthesis tool, design library, and synthesis scripts to ensure a fair and consistent evaluation.

As summarized in Table 6, our round-based implementations achieve substantial area reduction with respect to the state-of-the-art Ascon-128a designs [14]. In this case, our implementations under the *v1r1c*, *v2r1c*, and *v4r1c* design architectures have 39.99%,

Table 7: Operation-based state-of-the-art design comparison.

Function	Design	Variant	Area (μm^2)	Delay (ps)	Cycles	Latency (ns)	Power (μW)	Energy (pJ)
Encryption	[10]	<i>v1op1c_64b_59</i>	11404	2844	2861	8137	7582	61692
	Ours	<i>v1op1c_64b_28</i>	11651	3075	1373	4222	9894	41772
Decryption	[10]	<i>v1op1c_64b_59</i>	11646	2872	2861	8217	7450	61215
	Ours	<i>v1op1c_64b_28</i>	11968	2865	1373	3934	9187	36138
Unified Encryption/Decryption	[10]	<i>v1op1c_64b_59</i>	11622	3002	2861	8863	7449	63977
	Ours	<i>v1op1c_64b_28</i>	11977	2795	1373	3838	9663	37082
Unified Hash	[10]	<i>v1op1c_64b_59</i>	8459	2294	4263	9779	6067	59331
	Ours	<i>v1op1c_64b_28</i>	8871	2364	2031	4801	5908	28366
Unified Family	[10]	<i>v1op1c_64b_59</i>	11987	2950	2861	8440	7440	62793
	Ours	<i>v1op1c_64b_28</i>	12340	2907	1373	3991	9123	36413

33.38%, and 24.62% less area. The *v1r1c* variant reduces the latency by 21.42% due to its shorter critical-path, whereas the *v2r1c* and *v4r1c* variants increase the latency by 5.26% and 33.96%, respectively, mainly due to the different number of clock cycles in the reference designs. Such a difference arises because the state-of-the-art design [14] does not process the additional empty padded blocks that occur when the associated data and plaintext lengths are a multiple of 128. In contrast, our test vectors include these extra blocks in accordance with the latest NIST standard [19]. The energy consumption is reduced by 58.49% and 39.15% for the *v1r1c* and *v2r1c* variants, respectively, while the *v4r1c* variant shows 11.93% increase in energy consumption due to different number of clock cycles as explained before.

For the operation-based variants, the work of [10] is the only available state-of-the-art design that performs one 64-bit operation per clock cycle. Since its GitHub release [12] contains a bus-interfaced implementation, we implemented the *v1op1c_64b_59* variant as described in [10], which uses a 64-bit operation and 59 clock cycles to complete one permutation round for the NIST Ascon family, to enable a fair comparison. The *v1op1c_64b_59* variant is implemented for the Ascon family as shown in Table 7. For the Ascon unified hash, the reported cycle count, latency, and energy consumption are computed based on the Ascon-Hash256 and Ascon-XOF128. Likewise, for the Ascon unified family, the reported cycle count, latency, and energy are computed based on the Ascon-AEAD128.

As reported in Table 7, our 64-bit operation-based variant *v1op1c_64b_28* incurs a modest area overhead of 2.17%, 2.76%, 3.05%, 4.87%, and 2.94% when compared to the *v1op1c_64b_59* variant for the encryption, decryption, unified encryption/decryption, unified hash, and the Ascon unified family, respectively. Despite this, latency decreases significantly by 48.11%, 52.12%, 56.70%, 50.91%, and 52.71% across the same designs, primarily due to the reduced number of clock cycles required for completing one permutation round in our implementation. Energy consumption is also reduced by 32.29%, 40.97%, 42.04%, 52.19%, and 42.01%, across the same designs.

5 Conclusions

This work proposes two design architectures to explore the design space of the NIST-standard Ascon family. The round-based variants can implement the permutation block in 1, 2, and 4 rounds per one clock cycle, while the operation-based variants can execute one 64-bit and 32-bit operation per clock cycle in an optimized-cycle schedule of operations, thereby significantly reducing the latency and energy consumption when compared to the state-of-the-art designs. Experimental results show that the round-based designs are preferable when low latency and energy consumption are the highest priority, whereas the operation-based designs are the best

choice for applications with a low area requirement. In addition, the proposed unified designs support multiple Ascon functions with a modest hardware overhead by reusing the same permutation block and datapath resources. Overall, these alternative designs enable a designer to choose the one that satisfies the design requirements. As future work, we have been implementing the secure Ascon family against the side-channel analysis and fault injection attacks.

Acknowledgments

The authors thank Prof. Maria Eichlseder for providing the source codes of Ascon v1 [12]. This work was co-funded by the European Union and Estonian Research Council via project TEM-TA138, the Estonian Research Council Grant PRG2531, and Estonian Ministry of Defence under grant No 2-2/24/541-1.

References

- [1] Ray Beaulieu et al. 2015. The SIMON and SPECK Lightweight Block Ciphers. In *DAC*. 1–6.
- [2] Andrey Bogdanov et al. 2007. PRESENT: An Ultra-Lightweight Block Cipher. In *CHES*. 450–466.
- [3] Christoph Dobraunig et al. 2014. Ascon v1. CAESAR Competition.
- [4] Christoph Dobraunig et al. 2015. Ascon v1.1. CAESAR Competition.
- [5] Christoph Dobraunig et al. 2016. Ascon v1.2. CAESAR Competition.
- [6] Christoph Dobraunig et al. 2021. Ascon v1.2. Submission to NIST.
- [7] Mohamed El-Hadedy et al. 2023. RECO-ASCON: Reconfigurable ASCON Hash Functions for IoT Applications. *Integration* 93 (2023), 102061.
- [8] Islam Elsadek and Eslam Tawfik. 2025. Optimized and Reconfigurable Hardware Design for ASCON AEAD and Hash Standards. In *ISCAS*. 1–5.
- [9] Islam Elsadek and Eslam Yahya Tawfik. 2024. Efficient Programmable Architecture for LWC NIST FIPS Standard ASCON. In *ISDFS*. 1–5.
- [10] Hannes Groß et al. 2015. Suit up!—made-to-measure Hardware Implementations of Ascon. In *DSD*. 645–652.
- [11] Kashif Inayat. 2024. Ascon-Lightweight. <https://github.com/KashifInayat/ascon-lightweight>. Accessed: 2026-02-18.
- [12] ISEC, Graz University of Technology. 2016. Ascon_Hardware. https://github.com/isec-tugraz/ascon_hardware/tree/master/asconv1. Accessed: 2026-02-18.
- [13] Safiullah Khan et al. 2022. Area-Time Efficient Implementation of NIST Lightweight Hash Functions Targeting IoT Applications. *IEEE IoT-J* 10, 9 (2022), 8083–8095.
- [14] Safiullah Khan et al. 2024. Securing the IoT Ecosystem: ASIC-based Hardware Realization of Ascon Lightweight Cipher. *IJIS* (2024), 1–12.
- [15] Safiullah Khan, Wai-Kong Lee, and Seong Oun Hwang. 2022. Evaluating the Performance of Ascon Lightweight Authenticated Encryption for AI-Enabled IoT Devices. In *TRONSHOW*. 1–6.
- [16] Olof Kindgren. 2020. SERV—The Serial RISC-V CPU. <https://github.com/olofk/serv>. Accessed: Nov. 2025.
- [17] Ahmet Malal. 2025. High-Performance FPGA Implementations of Lightweight ASCON-128 and ASCON-128a with Enhanced Throughput-to-Area Efficiency. *Cryptology ePrint Archive* (2025).
- [18] Khai-Duy Nguyen et al. 2024. Asic Implementation of Ascon Lightweight Cryptography for IoT Applications. *IEEE TCAS-II* 72, 1 (2024), 278–282.
- [19] Meltem Sönmez Turan et al. 2025. *Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions*. NIST Special Publication 800-232. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-232
- [20] Xiangdong Wei et al. 2022. RECO-HCON: A High-Throughput Reconfigurable Compact ASCON Processor for Trusted IoT. In *SOC*. 1–6.